SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 — ASSESSMENT TOOL 2

Architecture Design Assignment

Course Code	**CSA1017**
Course Title	**Software Engineering**
CO Assessed	**CO2**
Assessment	**Assessment Tool 2 — Architecture Design Assignment**
Student Name	**Saruhaasini A**
Register Number	**192511400**
Department	**CSE**

AI-Powered Smart Textile Manufacturing Quality Inspection System

Code of Conduct

I, Saruhaasini A (192511400), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Saruhaasini A

1. System Requirements Analysis

1.1 System Objectives

The AI-Powered Smart Textile Manufacturing Quality Inspection System aims to replace slow, inconsistent manual fabric inspection with an automated, AI-driven platform. Its core objectives are to detect textile defects using computer vision, classify and score defects automatically, continuously monitor production-line quality, alert operators in real time, and generate analytics-driven quality reports — all while integrating with existing enterprise systems and remaining scalable as production volume grows.

1.2 Functional Requirements Influencing Architecture

- Continuous, high-throughput image capture and pre-processing from multiple production-line cameras.
- Low-latency AI inference for real-time defect detection, classification, and severity scoring.
- Real-time alerting to operators and managers when quality thresholds are breached.
- Asynchronous report generation and analytics aggregation independent of the real-time detection path.
- Secure, role-based access across multiple distinct user types (operators, inspectors, managers, administrators).
- Integration with external ERP/MES systems via well-defined APIs.

These requirements point toward an architecture that separates real-time, latency-sensitive processing from asynchronous, batch-oriented processing, while keeping each capability independently deployable and scalable — directly motivating the architectural style selected in Section 2.

1.3 Quality Attributes Influencing the Architecture

Quality Attribute	Architectural Driver
Scalability	The system must scale from a single pilot production line to multiple lines and factories without architectural redesign.
Performance	AI defect-detection inference must complete within sub-second latency per frame to keep pace with production-line speed.
Reliability	The system must continue functioning correctly under continuous, unattended operation across production shifts.
Availability	Core inspection functions must remain available even during temporary cloud connectivity loss (edge fallback).
Security	Production and quality data must be protected via encryption and role-based access control.
Maintainability	Individual capabilities (detection, classification, reporting) must be independently updatable without full-system redeployment.

2. Selection of Software Architecture

2.1 Architectural Styles Considered

Architectural Style	Strengths for This System	Limitations for This System
Layered Architecture	Simple, well-understood, clear separation of concerns.	Poor fit for independent scaling of AI inference vs. reporting; whole layers scale together.
Client-Server	Straightforward request/response model.	Not well suited to continuous, high-volume real-time image streams and asynchronous alerting.
Microservices	Independent scaling, deployment, and technology choice per service.	Alone, does not natively address real-time, asynchronous event propagation (e.g., alerts).
Event-Driven Architecture	Excellent for real-time, asynchronous processing of continuous image/defect streams.	Alone, lacks clear guidance on organizing business capabilities into deployable units.
Hybrid (Layered Edge-Cloud + Event-Driven Microservices) — Selected	Combines edge/cloud tiering, independent service scaling, and real-time event propagation.	Higher initial design and operational complexity than a single pure style.

2.2 Selected Architecture and Justification

Selected Architecture: Hybrid — Layered Edge-Cloud Architecture with Event-Driven Microservices

- A three-tier physical layering (Edge → Cloud → Client/External) separates where processing happens.
- Within the cloud tier, independently deployable microservices implement each business capability (detection, classification, alerting, reporting, access control, integration).
- An event bus (publish/subscribe message broker) connects services asynchronously, enabling real-time propagation of image-captured, defect-detected, and alert-raised events without tight coupling.

This hybrid style is the most suitable choice because the problem statement combines three distinct architectural pressures that no single classical style addresses cleanly: (1) physically distributed processing across shop-floor edge devices and cloud infrastructure, (2) a requirement for independently scalable, independently deployable capabilities as the system grows from one pilot line to many factories, and (3) a need for real-time, asynchronous propagation of detection and alert events without blocking the image-processing pipeline. Layering provides the edge-cloud physical separation; microservices provide independent scalability and maintainability per capability; and the event-driven backbone provides the low-latency, loosely coupled communication that real-time quality inspection demands.

3. Software Architecture Diagrams

Three complementary diagrams are provided to illustrate the proposed architecture from different perspectives: the overall logical structure, the physical deployment of components, and the runtime message flow.

3.1 High-Level Architecture Diagram

The diagram below shows the full logical structure: the presentation layer, the API gateway, the microservices layer, the event bus / message broker, the edge layer, the data layer, and the external ERP/MES system.

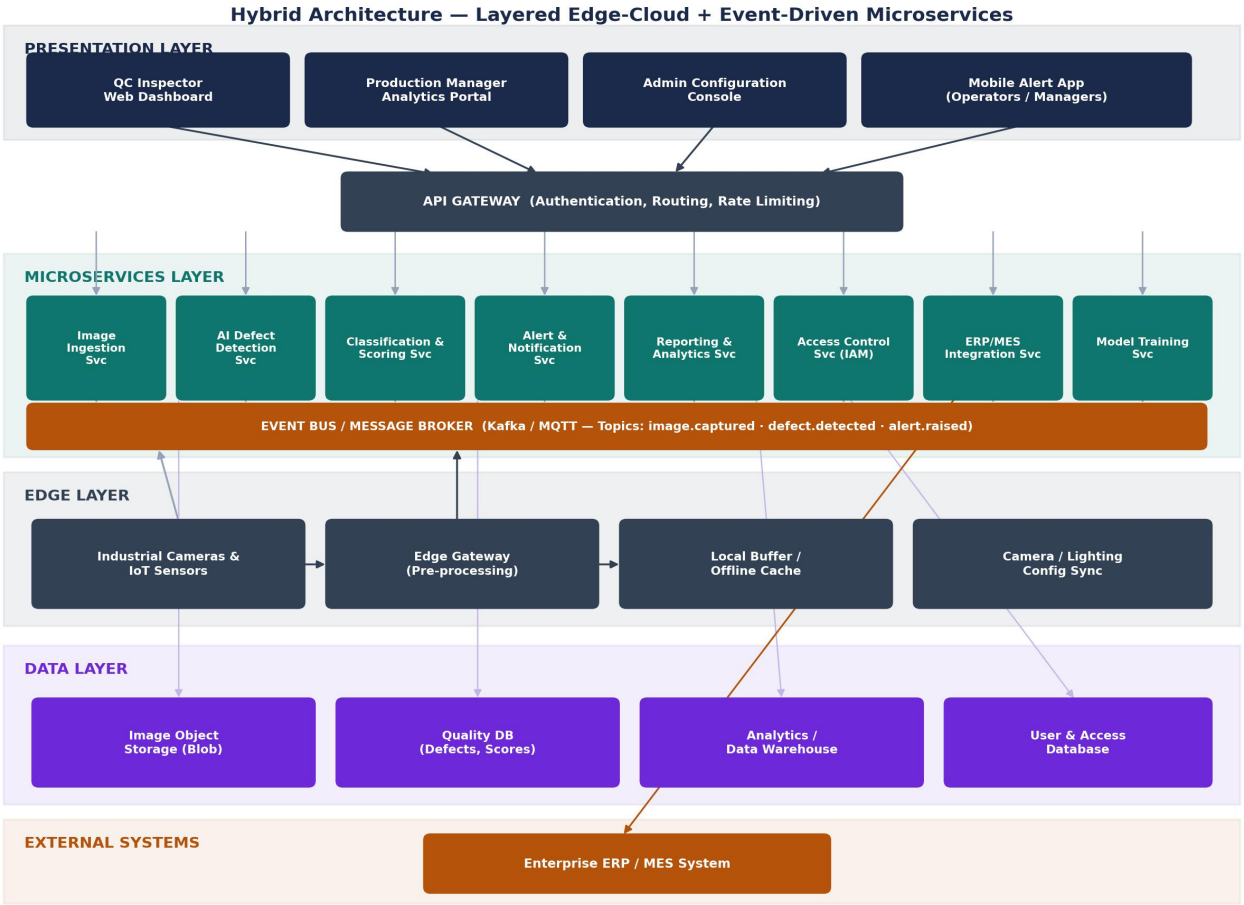

Figure 3.1 — High-level hybrid architecture: layered edge-cloud structure with an event-driven microservices core.

3.2 Deployment Diagram

The deployment diagram illustrates the physical distribution of components across nodes — the shop-floor edge gateway, the cloud Kubernetes cluster hosting the microservices and event bus, managed data services, client devices, and the external enterprise server — along with the communication protocols used between them.

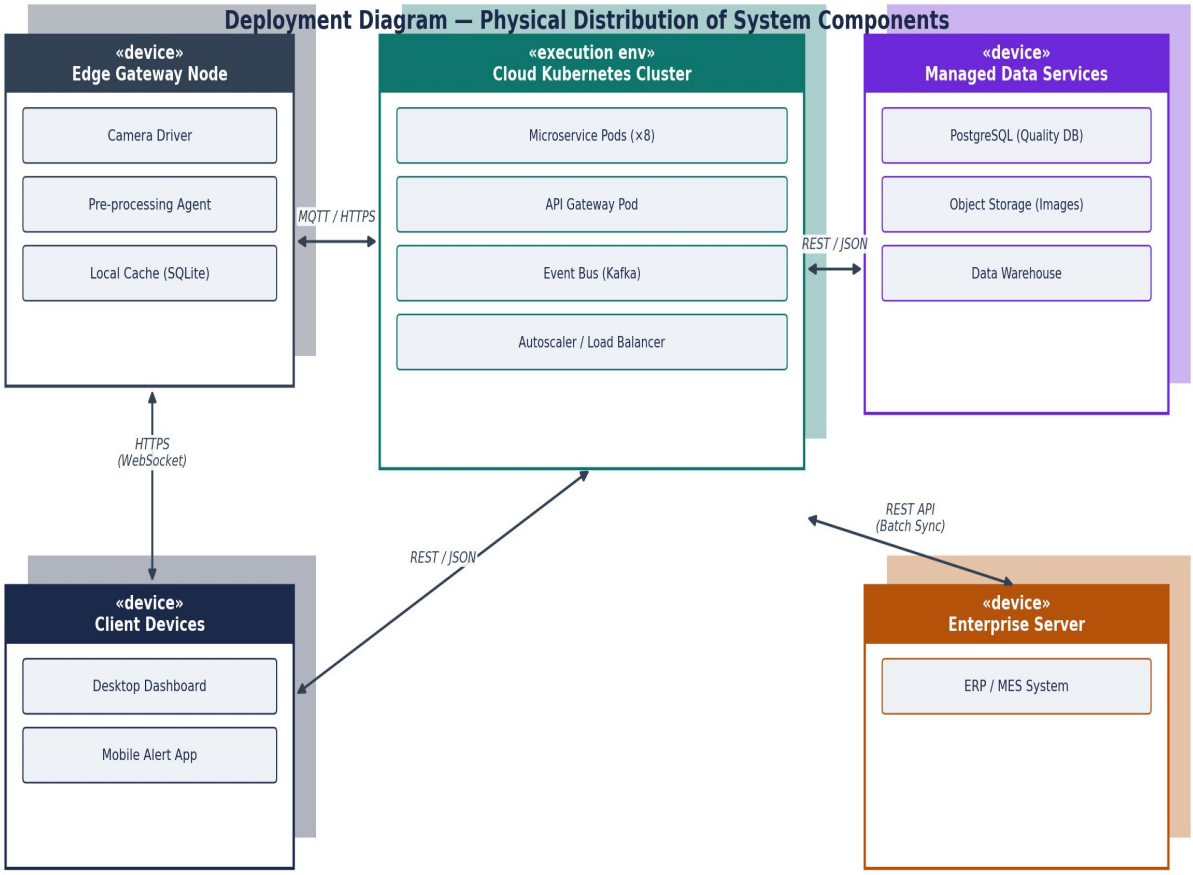

Figure 3.2 — Deployment diagram showing physical nodes and communication protocols.

3.3 Sequence / Workflow Diagram

The sequence diagram traces a single fabric image through the system end-to-end: from capture at the edge, through asynchronous detection and classification via the event bus, to real-time alerting and dashboard update for the QC inspector.

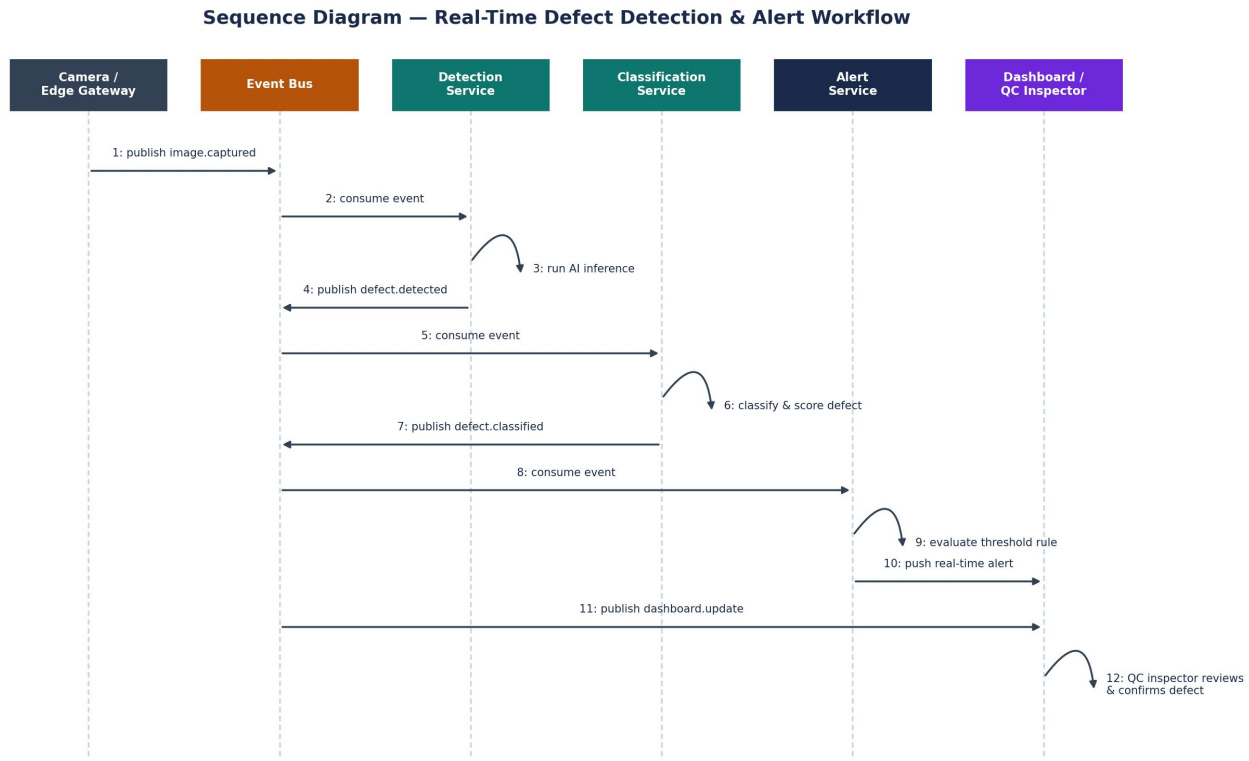

Figure 3.3 — Sequence diagram of the real-time defect detection and alert workflow.

4. Components and Their Interactions

4.1 Component Responsibilities

Component	Responsibility	Interaction / Protocol
Edge Gateway	Captures fabric images from cameras, performs local pre-processing (noise reduction, normalization), and buffers data during connectivity loss.	Publishes image.captured events to the Event Bus via MQTT/HTTPS.
API Gateway	Single entry point for all client requests; handles authentication, request routing, and rate limiting.	Routes HTTPS requests from client apps to the appropriate microservice.
Image Ingestion Service	Receives and stores incoming image streams; triggers the detection pipeline.	Consumes image.captured events; writes to Image Object Storage.
AI Defect Detection Service	Runs computer-vision inference to identify visual defects in fabric images.	Consumes ingestion events; publishes defect.detected events.
Classification &	Categorizes each defect and	Consumes defect.detected

Component	Responsibility	Interaction / Protocol
Scoring Service	assigns a severity score.	events; publishes defect.classified events.
Alert & Notification Service	Evaluates defect-rate thresholds and pushes real-time alerts.	Consumes classification events; pushes alerts via WebSocket/SMS to clients.
Reporting & Analytics Service	Aggregates quality data into shift-wise and daily reports and dashboards.	Consumes events asynchronously; reads/writes the Data Warehouse.
Access Control Service (IAM)	Manages user authentication, roles, and permissions.	Invoked by the API Gateway on every authenticated request.
ERP/MES Integration Service	Synchronizes quality data with the organization's enterprise systems.	Communicates with the external ERP/MES server via REST/JSON batch sync.
Model Training Service	Retrains the AI detection model using newly labelled defect data.	Reads historical data from the Data Warehouse; deploys updated models.

4.2 Overall Workflow

As illustrated in Figure 3.3, a fabric image begins its journey at the Edge Gateway, which publishes an image-captured event to the Event Bus rather than calling downstream services directly. This decoupling means the Detection Service, Classification Service, and any future subscriber (e.g., an analytics stream processor) can each independently consume the same event without the Edge Gateway needing to know they exist. Detection and classification results are themselves published as new events, allowing the Alert Service and Reporting Service to react in parallel — an alert can reach a machine operator within seconds of a defect being detected, while the same event is simultaneously and asynchronously folded into the daily quality report, with neither process blocking the other.

5. Discussion of Quality Attributes

The radar chart below summarizes a qualitative assessment of how well the proposed hybrid architecture satisfies each of the six quality attributes central to this assignment, followed by a detailed discussion of each.

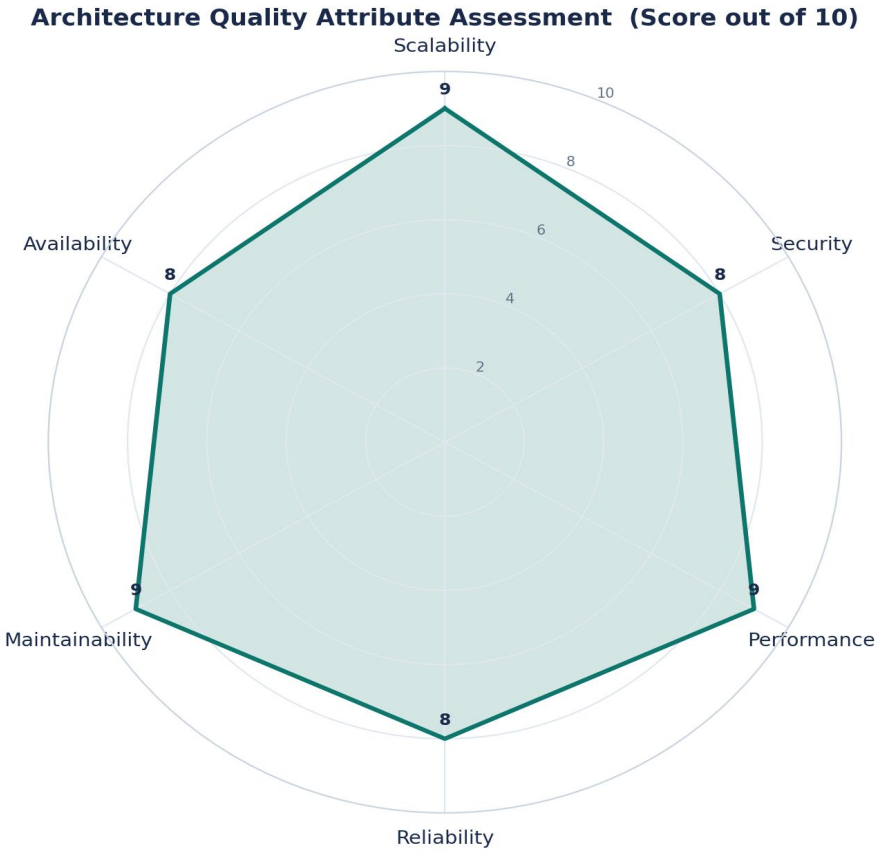

Figure 5.1 — Qualitative assessment of the proposed architecture against six key quality attributes.

Scalability

Because each microservice is independently deployable, the Detection Service — the most computationally intensive component — can be scaled horizontally (more inference pods) without scaling less-demanding services like Access Control. The event bus further decouples producers from consumers, so new production lines can be added simply by publishing additional image streams; no downstream service needs to know how many lines exist upstream.

Security

All client traffic passes through a single API Gateway, centralizing authentication and enabling consistent enforcement of role-based access control via the Access Control (IAM) Service. Data is encrypted in transit (TLS) between edge, cloud, and client tiers, and at rest within managed data services, limiting the impact of any single component's compromise.

Performance

Latency-sensitive detection and classification run as dedicated, independently scalable services directly on the event bus, avoiding contention with lower-priority, batch-oriented work such as report generation. This separation allows the system to meet the sub-second inference target even as reporting workloads grow.

Reliability

The event-driven design provides natural retry semantics: if the Classification Service is briefly unavailable, defect-detected events remain queued on the event bus rather than being lost, and processing resumes automatically once the service recovers. Local buffering at the Edge Gateway similarly protects against transient network interruptions between the shop floor and the cloud.

Maintainability

Each microservice owns a narrow, well-defined responsibility and can be updated, tested, and redeployed independently — for example, retraining and redeploying the AI model via the Model Training Service requires no changes to the Alert or Reporting services. This modular boundary also makes the system easier to reason about and onboard new developers onto.

Availability

Edge-local pre-processing and buffering allow core image capture to continue even during a cloud outage, with data synchronized once connectivity is restored. In the cloud tier, container orchestration (Kubernetes) automatically restarts failed service instances and load-balances across replicas, supporting the 99.5% uptime target during production hours.

6. Justification of Architectural Decisions

6.1 Rationale Summary

The hybrid layered edge-cloud and event-driven microservices architecture was chosen because it is the only option among those considered that simultaneously satisfies the system's physical distribution needs (shop floor vs. cloud), its independent-scaling needs (AI inference vs. reporting vs. access control), and its real-time communication needs (sub-second alerting) without forcing an artificial compromise between them.

6.2 Trade-offs Considered

Trade-off	Justification
Operational complexity vs. flexibility	A hybrid, multi-service architecture is harder to operate than a monolith, but this cost is accepted in exchange for independent scalability and fault isolation critical to a 24/7 production environment.
Eventual consistency vs. strong consistency	Asynchronous event propagation means the dashboard may lag real-world state by a few hundred milliseconds; this is acceptable given the real-time alerting path is prioritized separately and does not depend on dashboard consistency.
Infrastructure cost vs. resilience	Running a message broker and container orchestration platform adds infrastructure cost compared to a simple client-server deployment, but is justified by the reliability and scalability requirements of continuous industrial operation.
Development overhead vs. long-term agility	Defining service boundaries and event contracts upfront takes more initial design effort than a simpler layered build, but pays off as new capabilities (e.g., predictive analytics) can be added as new services without disturbing existing ones.

6.3 Support for Future Enhancements, Integration, and Long-Term Maintainability

- New capabilities — such as predictive maintenance analytics or AR-based defect visualization — can be added as new microservices that simply subscribe to existing events, without modifying the detection or classification pipeline.
- The API Gateway and event-driven integration pattern make it straightforward to onboard additional external systems (e.g., a supplier quality portal) alongside the existing ERP/MES integration.

- Because each service is independently deployable, the AI model can be retrained and redeployed frequently to keep pace with new fabric types or defect patterns, without any risk to the stability of unrelated services.
- The clear separation of edge, cloud, and data tiers means the system can scale from a single pilot production line to a multi-factory rollout by replicating the edge tier and horizontally scaling cloud services, with no fundamental architectural change required.

Conclusion

This architecture design analysis identifies a hybrid layered edge-cloud and event-driven microservices architecture as the most suitable design for the AI-Powered Smart Textile Manufacturing Quality Inspection System. The architecture directly addresses the system's real-time performance, scalability, reliability, and integration requirements while remaining maintainable and extensible for future enhancements, providing a solid technical foundation for subsequent implementation.